DEGREE PROJECT

Keystroke dynamics for student authentication in online examinations

Rebecka Mattsson

Computer Science and Engineering, master's level
2020

Luleå University of Technology
Department of Computer Science, Electrical and Space Engineering

LULEÅ UNIVERSITY OF TECHNOLOGY

Abstract

Biometrics are distinctive for each person, and can not be given away or hacked like a password. Keystroke dynamics is a behavioral biometric characteristic that can be used as a complementary authentication step [1]. In online examinations it is difficult to make sure that each student write their own work. Keystroke dynamics from these examinations could be used to detect attempted cheating.

To detect cheating attempts, a Gaussian Mixture Models with Universal Background Model (GMM-UBM) was implemented, and tested on benchmark data set recorded from online examinations written in free text. The use of a Universal Background Model (UBM) allows students to be enrolled using a limited amount of data, making the suggested approach suitable for the use case. The use of a GMM-UBM resulted in an Equal Error Rate (ERR) of 5.4% and an accuracy of 94.5%.

Preface

This thesis was written at BehavioSec. I would like to thank my external supervisor Julian Breitling for all of the discussions and guidance. I would also like to thank my internal supervisors Foteini Liwicki and Marcus Liwicki for helping me structure my work.

I have learnt a lot from working at BehavioSec, and would like to thank everyone there for always being open for discussions, and for making this a great experience for me.

Rebecka Mattsson

Contents

Chapter 1 – Introduction 5
1.1 Background . . . 5
1.2 Motivation . . . 5
1.3 Problem definition . . . 6
1.4 Delimitations . . . 6
1.4.1 Student platform . . . 6
1.4.2 Updating user models . . . 6
1.4.3 Features . . . 7
1.5 Ethical considerations . . . 7
1.6 Thesis structure . . . 7

Chapter 2 – Related work 9
2.1 Shared accounts . . . 9
2.2 Keystroke dynamics . . . 9
2.2.1 Benchmark data sets . . . 9
2.2.1.1 Predefined password . . . 10
2.2.1.2 Predefined sentences . . . 10
2.2.1.3 Free text . . . 10
2.2.2 Gaussian Mixture Model . . . 10
2.2.3 Prevent cheating in e-learning . . . 11
2.3 Summary . . . 11

Chapter 3 – Theory 13
3.1 Keystroke features . . . 13
3.2 Normalization . . . 13
3.3 Gaussian Mixture Model . . . 14
3.3.1 Expectation Maximization . . . 14
3.3.2 Universal Background Model . . . 15
3.3.3 Maximum A Posteriori adaptation . . . 16
3.4 Likelihood-ratio test . . . 16
3.5 Performance measurements . . . 17

Chapter 4 – Method 19
4.1 Method . . . 19

4.2 Optimization 20
4.2.1 Feature selection 20
4.2.2 Training UBM on another data set 20

Chapter 5 – Experimental setup 23
5.1 Programs and libraries 23
5.2 Data set 23
5.2.1 Data preprocessing and feature extraction 23
5.2.2 Background data 24
5.2.3 Test data 24
5.2.4 Second data set 24
5.3 Hyperparameters 24

Chapter 6 – Results 25
6.1 Student authentication 25
6.2 Feature selection 27
6.3 Training UBM on another data set 28
6.4 Summary 29

Chapter 7 – Discussion 31
7.1 Data set selection 31
7.2 Model selection 32
7.3 Performance 32
7.3.1 Comparison to similar work 32
7.3.2 Second data set for UBM 33
7.3.3 Free text 34
7.3.4 Hardware 34

Chapter 8 – Conclusions and future work 35

Acronyms

ACC Accuracy.

ANN Artificial Neural Network.

CPANN Counter-Propagation Artificial Neural Network.

DBN Deep Belief Network.

EM Expectation Maximization.

ERR Equal Error Rate.

FNR False Negative Rate.

FPR False Positive Rate.

GMM Gaussian Mixture Model.

GMM-UBM Gaussian Mixture Models with Universal Background Model.

LLR Log Likelihood Ratio.

MAP Maximum A Posteriori.

PCA Principal Component Analysis.

ROC Receiver Operating Characteristic.

SVM Support Vector Machine.

TNR True Negative Rate.

TPR True Positive Rate.

UBM Universal Background Model.

VB-GMM Variational Bayesian Gaussian Mixture Model.

Chapter 1

Introduction

The behavioral characteristics of a person is difficult to mimic and can not be given away like a password. Therefore, behavioral biometrics is a good complement to the more common forms of authentication. This can be used to verify a users identity in situations where a user might voluntarily give away their password.

1.1 Background

Common forms of authentication are either knowledge based or token based. Knowledge based authentication such as passwords can be given away, guessed or hacked, and is therefore not enough to ensure that only the genuine user has access. Tokens refer to a physical object such as a card or device, which could be lost or stolen. [1]

Biometrics are physiological characteristics that are distinctive for each person. Examples of biometrics are fingerprints, iris or face. The behavioral biometrics can refer to how a person speaks or types. These characteristics are difficult for an impostor to mimic and can not be given away or stolen. [1]

Therefore, biometric authentication is a good complement to the more common forms of authentication.

1.2 Motivation

When a student hands in their work online, there is no way for the teacher to be sure who actually wrote it. There is software to check for plagiarism, but there is still a risk that someone else wrote the assignment for the student. A password to the students account can easily be given away. Keystroke dynamics could be used to prevent attempted cheating by continuously authenticating a student based on how they type.

Online examinations can be used for several purposes. For example, in programming courses it could provide a setting that is closer to how it would be done in a work place.

A common form for programming examinations today is to write solutions on paper, which means that a lot of focus is on syntax that would normally be provided by a code editor. Also, it could make education more available for distance students, and thereby allow more people to get a higher education.

To be able to keep a high standard of education, it is crucial to be able to know what each student is capable of. Thus, continuous biometric authentication could improve the examination situation for students and make it easier for a teacher to set fair grades.

1.3 Problem definition

Current solutions for authentication using biometric dynamics are mostly focused on detecting fraud, meaning that the real user does not approve of a shared account. In the use case of cheating in e-learning, the user can voluntarily log in or give away their credentials. The impostor is more likely to know what to type and what to do, which makes it more difficult to detect abnormal activities. In this case, instead of just looking for anomalies, there will be a classification between a genuine user and an impostor.

The goal of this thesis is to develop a method that will be able to detect whether a text is written by the expected student or by an impostor. This will be done by extracting features from keystroke dynamics and using a Gaussian Mixture Model (GMM), along with a UBM, to classify sessions as either the genuine user or an impostor.

The results from the method will be evaluated by comparing them to previous similar solutions presented in chapter 2.

1.4 Delimitations

There are many factors that can be considered to identify a shared account. For this project, only behavioral biometrics in the form of keystrokes will be taken into account.

1.4.1 Student platform

To be able to access the keystroke information it is a requirement that the assignments are written in the assigned platform. The use of copying and pasting text will not be considered for this project. If such data is in the data set, it will be removed because of the small number of keystrokes registered. Implementing this student platform is also out of scope for this project.

1.4.2 Updating user models

After a user-specific model has been trained, the model will be locked, and will not keep updating. The new data that comes in will be classified using the saved model, but will not change the model in any way.

1.4.3 Features

The features considered will only involve the timing of how keys are pressed, there will not be any analysis of the content of the text. Neither will any other biometric dynamics, such as mouse dynamics, be considered.

1.5 Ethical considerations

By recording keystrokes, it is possible to recreate the content of the text written. Therefore, the setting of the situation should be assessed from an ethical point of view before this method is used. Aspects like the context of the text, how the data is stored, and who would be able to access it should be considered.

In this use case, the text will be from educational examinations, meaning that there should not be any data that would be considered sensitive. Still, data should be stored in a secure manner, and access should be limited. In the case of more sensitive data, one could choose not to store any key-codes, and only record timestamps.

1.6 Thesis structure

In the next section there will be a summary of related work that this thesis builds upon. In section 3 there will be theoretical explanations of the models and methods used, and section 4 will describe the method. Section 5 will describe the experimental setup including data processing and languages and libraries used. The results are presented in section 6, followed by discussion in section 7. Lastly, there will be conclusions and suggestions for future work.

CHAPTER 2

Related work

Previous research on authentication include analyzing keystroke dynamics for passwords and continuous authentication based on the use of an application. This chapter will summarize the state of the arts within authentication based on behavioral biometrics.

2.1 Shared accounts

There has been several research projects about detecting shared accounts, with different goals and approaches. The most relevant ones for this thesis are the ones that focus on biometric authentication, more specifically keystroke dynamics. The research that has been done has either been focused on detecting shared passwords, or continuous authentication by analyzing the use of an application.

2.2 Keystroke dynamics

Keystroke dynamics has been widely used in the field of biometric authentication. An issue with comparing methods in this area is the use of different data sets and settings. Aspects like the amount of data, the keyboards used and whether the data is collected in a controlled environment will affect the results. This section will describe both solutions from benchmark data sets, but also some methods that used their own data sets. Even for the results that can not be compared to others, it could still indicate whether a method is worth further investigation.

2.2.1 Benchmark data sets

To create comparable results, several benchmark data sets has been conducted. Three of them are described below along with results from various methods.

2.2.1.1 Predefined password

A widely used benchmark data set consists of data from 51 subjects, each typing the password ".tie5Roanl" 400 times. This data set was published along with a comparison of 14 different anomaly detectors. The three top performing algorithms were Manhattan, Nearest Neighbor and Outlier Count that had ERR of between 9.6% and 10.2%. [2]

This data set was used to assess the performance of the GMM-UBM and a deep learning approach using a Deep Belief Network (DBN). These two models has previously performed well within the field of voice recognition. The same method as in the previous experiment was used, to make the results comparable, and the results from both approaches outperformed the previous models. The GMM-UBM had an ERR of 5.5% and the DBN 3.5%. There was also a comparison between GMM-UBM and regular GMM, where the GMM achieved an ERR of 8.7%, which is better than all 14 models from the previous work, but not as good as the GMM-UBM. [3]

2.2.1.2 Predefined sentences

Another attempt of creating comparable results was made by collecting a data set of 136 million keystrokes, and publishing both the data set and the observations made. 168 000 volunteers entered 15 predefined sentences in an online typing speed test. It was found that rollover behavior, described in section 3.1, was strongly correlated to the typing speed. It was also found that faster typists usually make fewer substitution errors. [4]

2.2.1.3 Free text

There was another collection of keystroke data to provide an open data set for research purposes. This time, data was collected from undergraduate students while writing online exams in an introductory computer science course. There were three exams that each consisted of five essay questions. One was typed with both hands, one with only their left hand and the last with only their right hand. [5]

The data set from [5] was used in a competition to classify students. The best performing strategy used pairwise coupling by using 2 regression models and one prediction model in a multi-classifier architecture. An Artificial Neural Network (ANN) and Counter-Propagation Artificial Neural Network (CPANN) was used for regression models and a Support Vector Machine (SVM) was used as prediction model. A bottom up tree structure was used for the pairwise coupling. The classification accuracy for users typing with both hands ranged from 58.1% to 74.4%.

2.2.2 Gaussian Mixture Model

There has also been research on using a Gaussian Mixture Model with several components to allow the digraph patterns to fit more than one distribution. The experiment showed that GMM using 2 components instead of pure Gaussian enhanced the results by 93.8%,

resulting in an ERR of 0.08%. Adding more than 2 components decreased the results rapidly. The experiments were done on an open data set consisting of 30 users, where data has been collected in a desktop environment in a lab at Clarkson University. Each user completed 2 sessions, where the first one included survey questions that was answered in free text, and the other is a static-text typing process. [6]

A common problem in web services is that several users share an account to avoid paying a fee or giving up personal information. To solve the problem, a solution for detecting number of users in an account was proposed. It was assumed that a user's keystroke pattern form a cluster in Euclidean space and that the number of users could be estimated by the number of clusters. To determine the number of clusters, Variational Bayesian Gaussian Mixture Model (VB-GMM) was used. The case study was performed on 25 passwords typed by 16 users. When it came to deciding whether the account was shared or not, there was a 2% false alarm rate, and a 2% miss rate. When attempting to detect the number of users, there was an error of 7% . The conclusion was that more extensive testing is required, but that the results were promising enough to suggest that it is worth further investigation. [7]

2.2.3 Prevent cheating in e-learning

For detecting cheating in e-learning examinations, there was an attempt of identifying students using a One-class SVM. Data was collected from distance students at a university and consisted of a short identification string that each user had for authentication. There were 1387 keystroke signatures from 36 students used as positive data, and 2247 negative keystroke signatures from 20 other users. The method resulted in an average identification accuracy of 84% and a false positive rate average of 8.77%. [8]

2.3 Summary

Table 2.1 summarizes the results and settings from previous studies. If a benchmark data set from section 2.2.1 has been used, the value in the *Data* column will refer to the section where it is described.

Study	**Detector / Model**	**Data**	**Accuracy**
[2]	Manhattan Nearest Neighbor Outlier Count	2.2.1.1 2.2.1.1 2.2.1.1	90.4% 90% 89.8%
[3]	GMM GMM-UBM DBN	2.2.1.1 2.2.1.1 2.2.1.1	91.3% 94.5% 96.5%
[5]	Pairwise coupling in multi-classifier architecture	2.2.1.3	58.1% - 74.7%
[6]	GMM	30 users static and free text	99.92%
[7]	VB-GMM	16 users static passwords	98% for shared account detection, 93% for estimating nr of users
[8]	One-class SVM	56 users short static text	84%

Table 2.1: Summary of results and settings from previous solutions.

Chapter 3

Theory

The theory behind this method includes how keystroke dynamics are used as features, how Gaussian Mixture Model (GMM)s work, as well as methods for measuring performance.

3.1 Keystroke features

The features typically assessed in keystroke dynamics are key press and key flight, where the flight time can occur in several combinations [9]. Keystrokes are recorded as up and down events with a timestamp for each event. In the rest of this section, an up event will be denoted as U, and a down event as D.

In Figure 3.1, image (a) shows the duration of a single keystroke, which is the difference in time between its down and up event. In image (b) there are two sequential keystrokes, with 4 different key flight times. Since the first key is released before the next is pressed, there will be no rollover, meaning that all flight times will be positive. The case of rollover is demonstrated in image (c), where the second key's down event occurs before the first key's up event. Rollover means that the *UD* value will be negative. Ultimately, the features considered for this thesis are *keypress*, *UU*, *DD*, *UD* and *DU*.

3.2 Normalization

Min-max normalization maps each value to a given range, while preserving the relationships between the original values. Another method is Z-score normalization, which is based on the mean and standard deviation of a given set of values. The Z-score normalization is preferred when there is a large amount of outliers, while the min-max normalization could be more useful if the set of values has a small standard deviation. [10, p. 114]

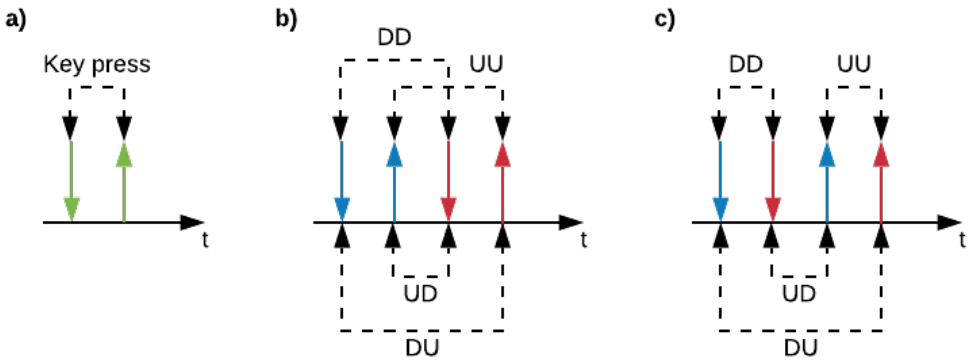

Figure 3.1: The figure shows key events and intervals for a) a single key, b) two keys without rollover and c) two keys with rollover.

3.3 Gaussian Mixture Model

A pure Gaussian fits data by a single mean and variance. When data belongs to more that one Gaussian distribution, a GMM can describe the data by using several components that each have their own mean, covariance and mixing probability. The use of several components allows for more complex data to be modeled than the use of a single Gaussian distribution [11]. A GMM can be described by

$$p(x_i|\theta) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x_i|\mu_k, \Sigma_k), \tag{3.1}$$

where each distribution in the mixture has a mean μ_k and a covariance matrix Σ_k.[12, p. 339]

The GMM is a probabilistic model, meaning that each point can belong to more than one cluster. Figure 3.2 illustrates an example of three separate Gaussian distributions. A mixture of these distributions would mean that a data point could belong to all three distributions, with different probabilities.

3.3.1 Expectation Maximization

To determine optimal parameters for a GMM, an Expectation Maximization (EM) algorithm can be used. The EM algorithm estimates parameters for probabilistic models with incomplete data. The attributes for this algorithm are:

- observed data (X)
- latent or missing data (Z)

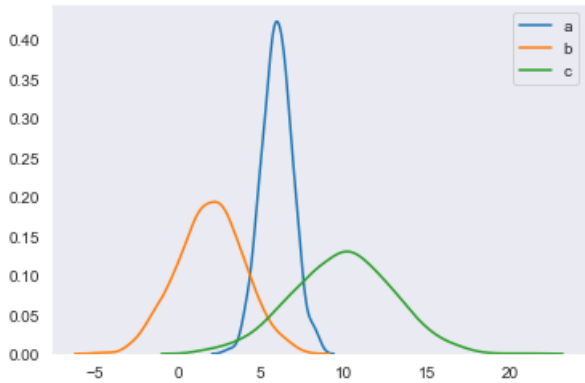

Figure 3.2: Example of three Gaussian distributions.

- unknown parameter (θ)
- likelihood function ($L(\theta; X, Z) = P(X, Z|\theta)$)

To optimize parameters, the EM algorithm alternates between the expectation (E) step and the maximization (M) step until it converges.

1. *E-step:* Calculate the expected value of the log likelihood function of $\theta(t)$, $Q(\theta|\theta(t))$, with respect to the conditional distribution of Z and given X. The expected value is given by
$$Q(\theta|\theta(t)) = E_{Z|X,\theta(t)}[\log L(\theta; X, Z)]. \tag{3.2}$$

2. *M-step:* Find the parameter that maximizes
$$\theta(t+1) = \arg_\theta \max Q(\theta|\theta(t). \tag{3.3}$$

   The M-step updates the values for density, mean and covariance to maximize the expected log-likelihood from the E-step. After each iteration, the probabilities for each data point is updated. [13, p. 12-13]

3.3.2 Universal Background Model

A UBM is a GMM trained on a large amount of user-independent data, which can be used to represent general user characteristics. These user-independent characteristics can then be compared to user-specific characteristics, using the likelihood-ratio test described in section 3.4, to decide whether to accept or reject a user. The UBM can also be used to train user-specific models by using Maximum A Posteriori (MAP) adaptation as described in section 3.3.3. [14]

3.3.3 Maximum A Posteriori adaptation

Instead of training each users GMM from scratch, the UBM can be used to initialize the parameters. By applying MAP adaptation to the means of the UBM, the model can be fitted more quickly and requires less data. [15]

For each mixture i in a UBM, consisting of M mixtures, the probability of a data sample x_t to belong to that mixture is given by

$$Pr(i|X_t) = \frac{w_i p_i(x_t)}{\sum_{j=1}^{M} w_j p_j(x_t)}. \tag{3.4}$$

Sufficient statistics are calculated from

$$E_i(x) = \frac{1}{n_i} \sum_{t=1}^{T} Pr(i|X_t) X_t, \tag{3.5}$$

where

$$n_i = \sum_{t=1}^{T} Pr(i|X_t). \tag{3.6}$$

The aim of performing MAP adaptation on the UBM is to adapt the means of the model for each specific user. The adapted mean is given by

$$\hat{\mu}_i = \alpha_i^m E_i(x) + (1 - \alpha_i^m)\mu_i, \tag{3.7}$$

where the adaptation coefficient is given by

$$\alpha_i^m = \frac{n_i}{n_i + r_m}. \tag{3.8}$$

The relevance factor r_m is a constant determining how much the observed training data should influence the model adaptation. [15]

3.4 Likelihood-ratio test

To determine whether a user is more likely to be the genuine user or an impostor, a likelihood-ratio test can be performed [14]. Given a session of keystrokes S, and a genuine user G, the user can be verified by deciding which of the following two hypothesis are more likely to be true:

$$H_0 : \text{S was written by G},$$

$$H_1 : \text{S was not written by G}.$$

H_0 is represented by a model λ_G that is dependent on the genuine user, while H_1 is represented by a background model $\lambda_{\overline{G}}$. If S has the feature vector X, the output Y(X) can be evaluated as follows:

$$Y(X) = \log p(X|\lambda_G) - \log p(X|\lambda_{\overline{G}}) \begin{cases} \geq \theta & \text{accept } H_0, \\ < \theta & \text{reject } H_0. \end{cases} \tag{3.9}$$

3.5 Performance measurements

Receiver Operating Characteristic (ROC) graphs are commonly used to visualize the performance of classifiers [16]. The performance is measured by the amount of positive and negative predictions in combination with their initial conditions. The evaluation of the method will result in values that are True Positive (TP), False Positive (FP), True Negative (TN) and False Negative (FN). The connection between the values, conditions and predictions can be visualized by a confusion matrix[13], as demonstrated in table 3.1.

	Condition positive	Condition negative
Prediction positive	TP	FP
Prediction negative	FN	TN

Table 3.1: Demonstration of a confusion matrix showing the connections between TP, FP, TN and TP.

The False Negative Rate (FNR), True Negative Rate (TNR), True Positive Rate (TPR) and False Positive Rate (FPR) are calculated as follows.

$$TPR = \frac{TP}{TP + FN} \tag{3.10}$$

$$FPR = \frac{FP}{FP + TN} \tag{3.11}$$

$$TNR = \frac{TN}{FP + TN} \tag{3.12}$$

$$FNR = \frac{FN}{TP + FN} \tag{3.13}$$

The Accuracy (ACC) is given by

$$ACC = \frac{TP + TN}{TP + TN + FP + FN}. \tag{3.14}$$

A ROC curve is given by plotting the FPR against the TPR. [16]

The ERR is given by the intersection of the FNR and FPR when plotted with a varying decision threshold. There is a trade-off between FNR and FPR, where it can sometimes be preferred to have a higher FPR, to prevent genuine users from being rejected. However, in high security applications it could be preferable to have a higher FPR. Thus, the ERR could be used in combination with FPR and FNR to find the threshold that is optimal for each specific use case. [9]

Chapter 4

Method

The method is based on Gaussian Mixture Models with Universal Background Model (GMM-UBM), which will be described in more detail in this chapter. Then, there will also be two optimization strategies presented.

4.1 Method

After preprocessing the data and extracting features, the method consists of 3 parts:

1. Development
2. Enrollment
3. Verification

In Figure 4.1 the development part refers to training the UBM with user-independent data. After that, the user-dependent data is used to enroll users by creating user specific GMMs. The GMMs are acquired by applying MAP adaptation on the UBM. In the verification part, test data is compared to both the GMM and a user specific GMM to get a Log Likelihood Ratio (LLR). The LLR is then compared to a predefined threshold to determine whether the test user is an impostor or the genuine user.

Development The UBM was trained on a large amount of user independent data. The data that did not belong to the assigned test users was used as background data for the UBM.

Enrollment To enroll a student, a model was adapted from the previously trained UBM. The means of the model was adapted according to the formulas defined in section 3.3.3. The models acquired for each student were stored to be used for verification.

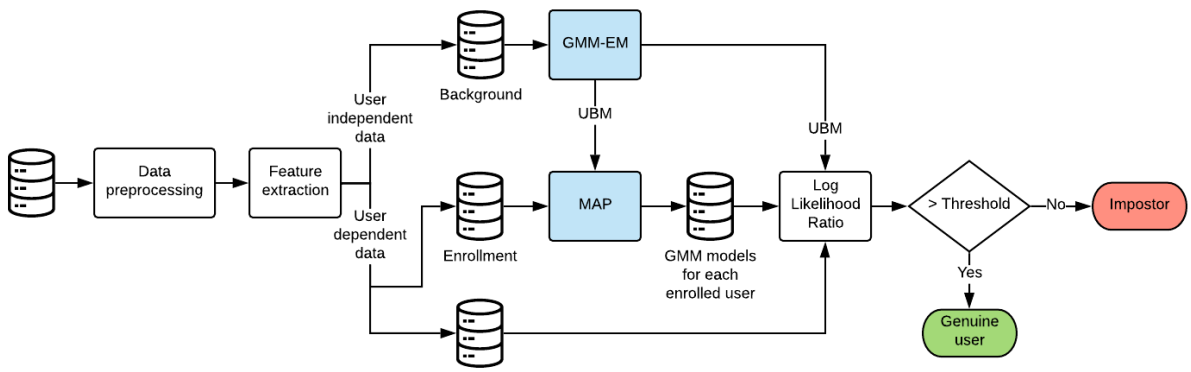

Figure 4.1: An overview of the method.

Verification To verify a user, the likelihood-ratio test described in section 3.4 was used. The threshold θ was varied to find the value that optimized the performance.

4.2 Optimization

After the implementation of the initial method, some additional approaches was attempted to optimize the performance of the model.

4.2.1 Feature selection

Different combinations of the features previously described were tested to optimize the performance. By removing one feature at a time, conclusions could be drawn about which features are most distinctive between users in this setting.

4.2.2 Training UBM on another data set

Since the UBM requires a large amount of data, another data set was used to provide additional data. Then, users from the original data set could be enrolled and verified in the same way as in the original method.

To have the UBM adapted for the test data, but reduce the amount of data needed, MAP adaptation was used on the UBM trained on the second data set. The means were then adapted with the background data used in the original approach. For this method, the test users were the same as in the original approach. Figure 4.2 illustrates this optimization strategy.

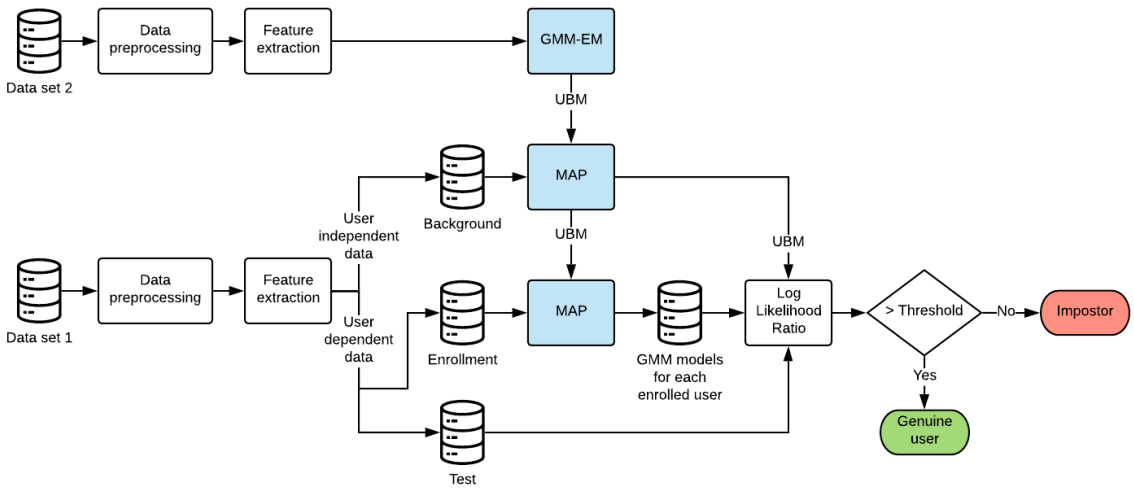

Figure 4.2: An overview of the method using a second data set.

Chapter 5

Experimental setup

This chapter will describe the settings and tools used for implementing and evaluating the presented method.

5.1 Programs and libraries

Several libraries were used for implementing and evaluating the method. Sklearn [17] was used both for preprocessing data and building Gaussian Mixture Models. NumPy [18] and Pandas [19] were used for data structures.

5.2 Data set

The entries where users typed with both hands were taken out from the data set described in section 2.2.1.3. It consists of 81 users answering 5 essay questions each. Since the UBM requires a sufficient amount of data, the performance of the method was evaluated with different ratios of background data and test users. The users that are not used in the background data were enrolled and used for testing.

5.2.1 Data preprocessing and feature extraction

The data was divided into digraphs and flight times were calculated. Then, outliers were removed by setting an upper and lower quantile limit for each feature. The limits were set by plotting and analyzing the distribution of the data.

Both min-max scaling and z-score normalization, described in section 3.2, were tested to determine which was most suitable for this data set.

5.2.2 Background data

The background data used for training the UBM is user-independent. Therefore, data from the users with the least amount of keystrokes each was chosen for this task, to make sure that the test users had enough data for both enrollment and verification.

5.2.3 Test data

The test data consists of the data that was left after taking out the background data for the UBM. The data was then divided for enrollment and verification. The data was divided into sessions consisting of 200 keystrokes each, where 10 sessions were used for enrollment and 10 for verification. When verifying enrolled users, 10 sessions from randomized impostors were scored. Thus, each test user had 10 positive and 10 negative samples for evaluation.

5.2.4 Second data set

For the optimization strategy described in section 4.2.2, another data set was used as background data for the UBM. The benchmark data set described in section 2.2.1.2 was preprocessed in the same way as the initial data set. Due to hardware and time limitations the first 10 million, out of the 136 million available, keystrokes were used.

5.3 Hyperparameters

The number of components for the GMMs, including the UBM, is a hyperparameter that needs to be set depending on the amount of data available. If there is an insufficient amount of data, in combination with a large number of components, the predictions made in the MAP adaptation will not work. Therefore, the number of components will vary between 16, 32, and 64 to find the optimal settings for this method.

CHAPTER 6

Results

This chapter will present the results for both the initial method described in section 4.1 and for the optimization strategies described in section 4.2.

6.1 Student authentication

Both min-max normalization and z-score normalization was tested for pre-processing of the data, where the min-max normalization performed best. Therefore, all results presented in this chapter were acquired from data that was normalized using min-max normalization.

The test was performed with varying ratio for background data and test users. The results are listed in Table 6.1. The maximum number of components possible for the data available was used.

Nr of test users	**Nr of componenets**	**ACC**
65	32	78.8%
57	32	85.6%
49	64	92.0%
41	64	93.7%
33	64	92.1%

Table 6.1: Results for varying ratio of background data and test users.

The best performing setting was the one with 41 test users and a UBM trained using 64 components. As seen in Figure 6.1 the FPR and FNR intersects when the threshold is set to 0.11. The ERR was calculated to be 6.6% at this threshold, which corresponds to an accuracy of 93.7%. Figures 6.2 and 6.3 shows the ROC curve and confusion matrix.

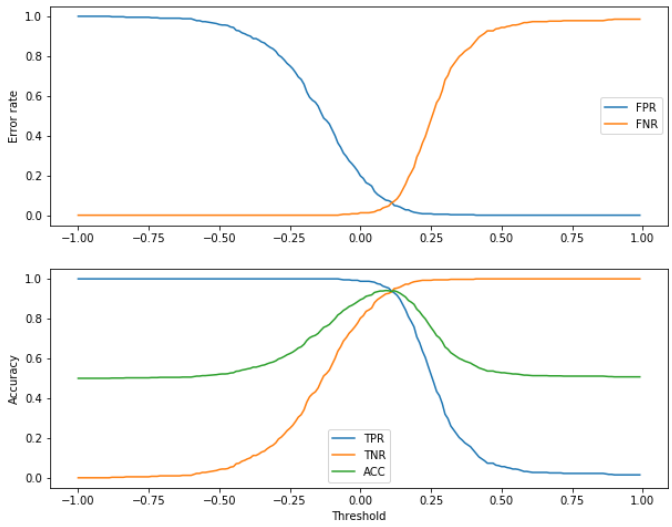

Figure 6.1: Results for varying threshold.

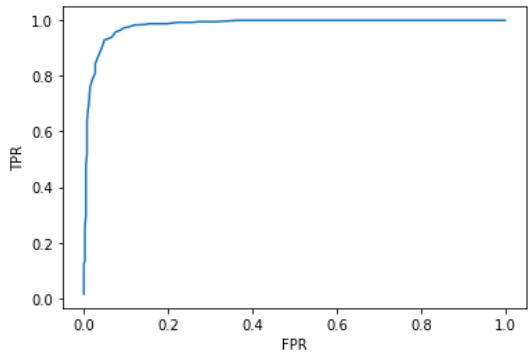

Figure 6.2: ROC curve showing the trade-off between FPR and TPR.

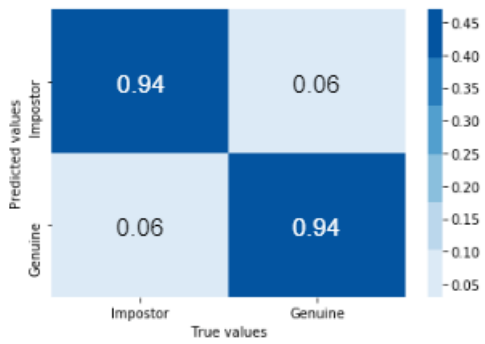

Figure 6.3: Confusion matrix.

6.2 Feature selection

To decide which combination of features are most distinctive for a user, several combinations were tested and compared. To make the results comparable to the results presented in section 6.1, the same settings were used. Therefore, 41 test users were used, along with 64 components. The results are listed in Table 6.2, where the first row displays the results from section 6.1.

Features	**ACC**
keypress, UU, DD, DU, UD	93.7%
keypress, UU, DD, DU	93.7%
keypress, UU, DD, UD	92.1%
keypress, UU, DU, UD	93.9%
keypress, DD, DU, UD	94.5%
UU, DD, DU, UD	89.3%

Table 6.2: Results for different combinations of features.

The best performing combination was achieved by removing the feature *UU*, which made the ERR drop from 6.6% to 5.4%, and the accuracy increased from 93.7% to 94.5%. These results are demonstrated in Figure 6.4, 6.5 and 6.6.

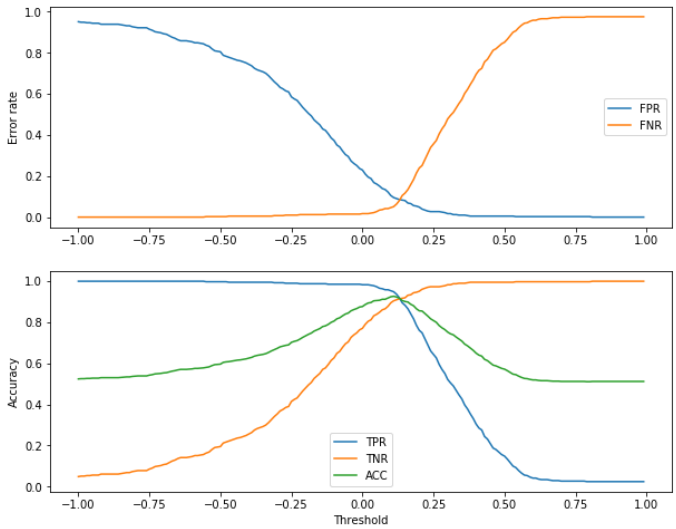

Figure 6.4: Results for varying threshold.

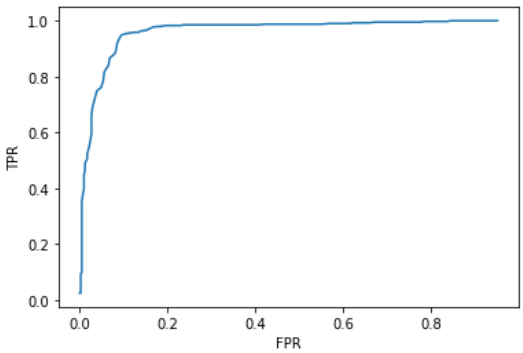

Figure 6.5: ROC curve showing the trade-off between FPR and TPR.

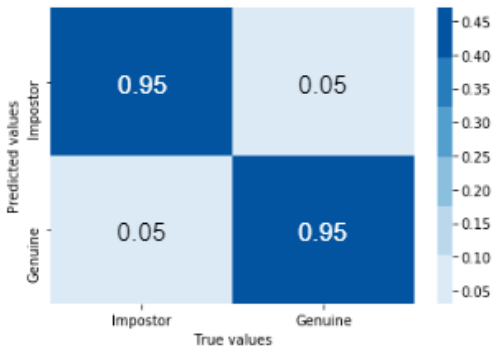

Figure 6.6: Confusion matrix.

By using less than 4 features, there was not enough data for the suggested method to work.

6.3 Training UBM on another data set

By training the UBM on another data set, all users in the initial data set are available as test users. To be able to compare the results, the test was performed both with the original test users, and with all users as test users.

When training the UBM on only the second data set, no users could be properly authenticated. The UBM was then improved by applying MAP adaptation with the background data used for the initial UBM. For the use of both data sets, 41 users was assigned as test users, and the rest was used for background data. With both data sets used for the UBM with 32 components, there was an ERR of 14%, which corresponds to an accuracy of 86%.

6.4 Summary

The results from the three approaches presented in this chapter are summarized in Table 6.3. The method from section 6.2 performed best, which was accomplished by removing the feature *UU*, leaving *keypress*, *DD*, *UD* and *DU*. It was also found that a higher number of components for the UBM gives better results, but that the amount of data available limits the number of components that can be used.

Method	**Features**	**Nr of components**	**ACC**
6.1	keypress, UU, DD, DU, UD	64	93.7%
6.2	keypress, DD, DU, UD	64	94.5%
6.3	keypress, UU, DD, DU, UD	32	86.0%

Table 6.3: Summary of results.

Chapter 7

Discussion

This chapter will discuss the choices made for the presented method, and how it performs compared to previous solutions.

7.1 Data set selection

The data set used was collected from online exams written by undergraduate students in an introductory computer science course [5]. Each exam collected free text answers from five essay questions. There were three exams, where the first one was the only one where students typed normally with both hands. For the second exam, students were instructed to type only with their left hand, and for the last they were to type only with their right hand. For the use case of this thesis it is most likely that students will type with both hands. Therefore, it was decided to only use data from the first exam.

The data set from [5] was decided to be suitable for this thesis based on two factors. First, it is a public data set collected for research purposes, meaning that it is possible to compare the presented solution to others that have used the same data set. Thus, it makes it easier to evaluate the method and open up for future research within the area. Secondly, the data set was collected in the same kind of setting that the use case for this solution would have. On the contrary of [2] and [4], the data was collected from free text answers in a real examination environment, meaning that the typing behaviour is closer to how a student actually behaves in that situation. Staged settings with predefined passwords or sentences might affect the users behaviour since they have to memorize what to type and might type words they would not normally use.

For the optimization strategy described in section 4.2.2, the data set from [4] was used as a second data set for the UBM. The main reason for choosing this data set was that there was a large amount of data available. Also, keystrokes from sentences was assumed to be more similar to free text than keystrokes from passwords. A more optimal solution would have been to have another data set collected in a similar setting, with keystrokes

from free text, but this was not available.

7.2 Model selection

From the literature study there were two methods that performed best on identifying users with keystroke dynamics, GMM-UBM and DBN [3]. The two methods both solve a classification problem with two possible outcomes, either genuine user or imposter.

The use of GMM allows features to be described by more than one distribution, which increases the performance for user authentication with keystroke dynamics [6]. In the GMU-UBM approach, a UBM is trained by applying the EM algorithm on data from several users. The UBM creates a relatively general profile, meaning that most users will have a quite high probability of belonging to that class. Each genuine user also has a GMM that is adapted from the UBM using MAP estimation to fit the specific users behaviour. The user-specific GMM will have a smaller variance and other users will be less probable to fit in to that profile. So if a genuine user is compared to the model, it might fit in partially in the UBM class, but will be likely to have a remarkably higher score when compared to its own class. The adaption from the UBM also means that less data is required from each specific student. This is beneficial to the use case since the model should be able to work early on in a course, without each student having to generate a lot of training data.

A problem with deep learning is that it requires a large amount of data and time to train the model. In the use case of detecting cheating in e-learning it could be difficult to collect enough data for each student. For the solution to fill its purpose, the model needs be able to perform on limited amount of data and be quick to train. Transfer learning can be used as supervised pre-training to learn the general characteristics of the data [20, p. 538]. In this scenario, a transfer learning approach could have the same benefit as the UBM, meaning that a previously trained model can be used to initialize a new users model. However, there is a risk that parameters will need extensive tuning to perform well, which is not suitable for the use case. Therefore, the GMM-UBM approach was decided to be used for this project.

7.3 Performance

This section will discuss the performance of the presented method in comparison to previous work as well as the optimization strategies described in section 4.2. It will also discuss some factors that might have affected the performance.

7.3.1 Comparison to similar work

In previous solutions the focus has mainly been on anomaly detection or detecting fraud. Many solutions focus on detecting shared password, while the research done on free text

is limited. A difficulty with free text is that the user might hesitate while deciding what to type, and the key sequences are not as rehearsed as when typing a password. This could make the keystroke dynamics less distinctive, and the amount of outliers could increase if the user stops typing to think about what they should write.

The best results achieved with the presented method had an ERR of 5.4% and an accuracy of 94.5%. Those results are better than the other attempt of detecting cheating in e-learning, where the resulting accuracy was 84% [8]. However, since they used their own data set of short static text, the results are not entirely comparable.

The data set used in this thesis has been previously used in a competition [5]. The participants were asked to authenticate users from text typed with both left, right and both hands. Only the results from text typed with both hands are comparable with the presented solution. The winning solution had an accuracy of $58.1\% - 74.7\%$ for both hands. For this competition, there was data from 64 users available, which could be because those users completed all three parts of the data collection. Since only one of the parts is used for this thesis, there was 81 users available. To make the results as comparable as possible, this accuracy should be compared to the results where 20% of the users was used as background data, leaving 65 test users. For these settings, the accuracy was 78.8%, which is an improvement.

Thus, the presented solution performed better than previous similar solutions, and the goals presented in the problem definition were fulfilled. However, it should be taken into account that the other solutions had some differing settings, which could explain the difference in accuracy.

7.3.2 Second data set for UBM

For the method described in this thesis, a large amount of data is required to train the UBM, which leaves less data for enrollment and testing. To solve this problem, there was an attempt made to train the UBM on another data set, which would be more suitable in the specified use case where background data could be difficult to acquire.

When using only the second data set for the UBM, it differed too much from the enrollment data, making it impossible to perform MAP adaptation since the predictions were all zero. Another attempt was made by performing the MAP adaptation on the UBM with the original background data. The use of both data sets made the method work, but the results were still better in the original method.

Using a second data set for the UBM did not improve the performance of the method. One of the reasons might be that the settings in which the second data set was acquired was too different from the test set. Typing pre-defined sentences means that the typing behaviour will depend on factors like how familiar the typist is with the words used and how much of the text can be memorized at once. Also, when typing free text, there are factors like how much a typist changes their mind on what to write and moves the cursor back and forth in the text. Thus, there are many factors that differ in typing situations,

and therefore this approach would need a data set that was acquired in a more similar setting.

7.3.3 Free text

The behaviour when typing free text differs from writing short static words such as passwords. A password is usually memorized and the typing behaviour is rehearsed. When typing free text it is common to stop to think about what to write, meaning that the typing speed varies between sentences. This leads to large variations in key flight times, which makes it difficult to preprocess the data and to know how much of the extreme values that can be removed without loosing distinctive behaviour.

More extensive research on behavioral biometrics in free text could lead to more insight in how to properly preprocess data from free text. Additional features that are not dependent on timing of keystrokes could also be beneficial in this scenario. This could be features like how much a user mistypes and uses the backspace key.

7.3.4 Hardware

There are some factors that could affect the result in a real world setting. Since this method is developed for remote examinations, the users are likely to use their own computers. This means different hardware, which could affect the results both to the better and to the worse. If an imposter uses a different computer than the genuine user, that could result in a more distinctive difference between the two users. An issue could be if the genuine user has been enrolled after using a certain keyboard, and then starts using a new keyboard. Both the difference in hardware, and the fact that the user is not used to the new one, could make the user appear like an impostor. To avoid this, one could update the users GMMs continuously.

Chapter 8

Conclusions and future work

The method described in this thesis resulted in an ERR of 5.4% and an accuracy of 94.5%. The use of a UBM allows students to be enrolled using a limited amount of data, making this approach suitable for the use case.

The method performed better than previous similar solutions and the goal of the thesis was achieved. However, that does not mean that the solutions is ready to be used in production. The method could still be improved, but will most likely never reach 100% accuracy due to factors like hardware limitations or the inconsistent flight times that can occur in free text. Therefore, the solution should not be the only factor to make decisions about whether a student has attempted to cheat. Instead, it could be used as a warning along with factors like how a student has performed in other assignments. For example, instead of accusing a student of cheating right away, the warning could lead to a discussion with the student to see if they seem to understand what they claim to have written.

The current solution assumed that there is already a platform for students to write their examinations in, where keystrokes can be recorded. This would need to be implemented before this solution could be used in production. Also, to reduce the impact from changing hardware or improving typing skills, user models could be continuously updated. This could be done by applying the MAP adaptation on the user model to adapt the means according to the new data. A challenge with this method is to acquire a sufficient amount of background data, that is similar enough to the user specific data, to train a well working UBM.

Another way to improve this method could be to consider features like how often a user mistypes and goes back to change text. In that case, it could also be interesting to see how the user navigates in the document, if keys or mouse is used.

An alternative method could be to detect shared accounts by estimating the number of users, in a given set of sessions, instead of authenticating each user. Detecting shared accounts could be considered more efficient and scalable. This would also mean that no initial data is required from each student, which could be difficult to acquire in a real

world setting. A previous attempt has been made with promising results [7]. However, this method would need to be tested more extensively, with a larger data set, to make comparable results.

References

[1] P. S. Teh, A. B. J. Teoh, and S. Yue, "A survey of keystroke dynamics biometrics," *The Scientific World Journal*, vol. 2013, pp. 1–24, 2013.

[2] K. S. Killourhy and R. A. Maxion, "Comparing anomaly-detection algorithms for keystroke dynamics," in *2009 IEEE/IFIP International Conference on Dependable Systems & Networks*, IEEE, June 2009.

[3] Y. Deng and Y. Zhong, "Keystroke dynamics user authentication based on gaussian mixture model and deep belief nets," *ISRN Signal Processing*, vol. 2013, pp. 1–7, 2013.

[4] V. Dhakal, A. Feit, P. O. Kristensson, and A. Oulasvirta, "Observations on Typing from 136 Million Keystrokes," in *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems (CHI '18)*, ACM, 2018.

[5] J. V. Monaco, G. Perez, C. C. Tappert, P. Bours, S. Mondal, S. Rajkumar, A. Morales, J. Fierrez, and J. Ortega-Garcia, "One-handed keystroke biometric identification competition," in *2015 International Conference on Biometrics (ICB)*, IEEE, May 2015.

[6] H. Ceker and S. Upadhyaya, "Enhanced recognition of keystroke dynamics using gaussian mixture models," in *MILCOM 2015 - 2015 IEEE Military Communications Conference*, IEEE, Oct. 2015.

[7] S. seob Hwang, H. joo Lee, and S. Cho, "Account-sharing detection through keystroke dynamics analysis," *International Journal of Electronic Commerce*, vol. 14, pp. 109–126, Dec. 2009.

[8] T. Eude and C. Chang, "One-class SVM for biometric authentication by keystroke dynamics for remote evaluation," *Computational Intelligence*, vol. 34, pp. 145–160, Aug. 2017.

[9] S. P. Banerjee and D. Woodard, "Biometric authentication and identification using keystroke dynamics: A survey," *Journal of Pattern Recognition Research*, vol. 7, no. 1, pp. 116–139, 2012.

[10] J. Han and M. Kamber, *Data mining: concepts and techniques*. Elsevier, 3rd ed ed., 2012.

[11] R. Chellappa, A. Veeraraghavan, N. Ramanathan, C.-Y. Yam, M. S. Nixon, A. Elgammal, J. E. Boyd, J. J. Little, N. Lynnerup, P. K. Larsen, and D. Reynolds, "Gaussian mixture models," in *Encyclopedia of Biometrics*, pp. 659–663, Springer US, 2009.

[12] K. P. Murphy, *Machine learning: a probabilistic perspective*. Adaptive computation and machine learning series, MIT Press, 2012.

[13] M. Awad and R. Khanna, *Efficient learning machines: theories, concepts, and applications for engineers and system designers*. The expertâs voice in machine learning, Apress Open, 2015.

[14] D. Reynolds, T. Hopper, M. Rejman-Greene, J. Li, X. Yang, X. Tao, and J. Tian, "Universal background models," in *Encyclopedia of Biometrics*, pp. 1349–1352, Springer US, 2009.

[15] D. A. Reynolds, T. F. Quatieri, and R. B. Dunn, "Speaker verification using adapted gaussian mixture models," *Digital Signal Processing*, vol. 10, pp. 19–41, Jan. 2000.

[16] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, pp. 861–874, June 2006.

[17] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[18] T. E. Oliphant, *A guide to NumPy*, vol. 1. Trelgol Publishing USA, 2006.

[19] W. McKinney *et al.*, "Data structures for statistical computing in python," in *Proceedings of the 9th Python in Science Conference*, vol. 445, pp. 51–56, Austin, TX, 2010.

[20] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016. http://www.deeplearningbook.org.